Отчёт по лабораторной работе №5

Имитационное моделирование

Самарханова Полина Тимуровна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Выполнение лабораторной работы **7**

3.1 Код модели 7

3.2 Базовый эксперимент 8

3.3 Анимация процесса 11

3.4 Итоговый отчет 13

4 Выводы **16**

Список иллюстраций

3.1 создание файла 7
3.2 код 8
3.3 создание папок и файла 8
3.4 код 9
3.5 установка пакетов 9
3.6 запуск 9
3.7 график1 10
3.8 график2 11
3.9 файл 11
3.10 код 12
3.11 установка пакетов 12
3.12 запуск 12
3.13 анимация 13
3.14 файл 13
3.15 код 14
3.16 запуск 14
3.17 графики 15

Список таблиц

1 Цель работы

Изучить моделирование задачи обедающих философов с использованием сетей Петри для анализа и предотвращения deadlock (взаимной блокировки). Реализовать две модели: классическую (с возможностью deadlock) и модифицированную (с арбитром), выполнить их стохастическое и детерминированное моделирование, визуализировать результаты и создать анимацию процесса.

2 Задание

- Создать рабочий каталог для всего курса.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
  - чистый код;
  - jupyter notebook;
  - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Выполнение лабораторной работы

3.1 Код модели

Перехожу в каталог лабораторной работы 5 и создаю папку src и файл Dining-Philosophers.jl (рис. 3.1).

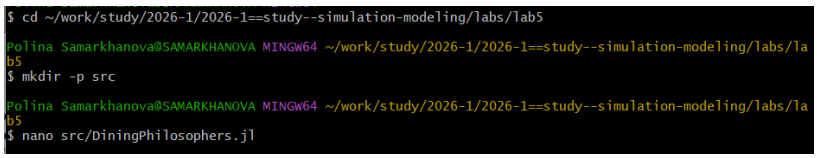

Рисунок 3.1: создание файла

В открывшемся окне вставила код из методички(это только модуль с моделью, запуск не нужен) (рис. 3.2).

```
        end
        t += dt
        if t <= tmax
            push!(times, t)
            push!(states, copy(u))
        end
    end

    df = DataFrame(time = times)
    for i = 1:net.n_places
        df[!, String(net.place_names[i])] = [s[i] for s in states]
    end
    return df
end

# Обнаружение deadlock

function detect_deadlock(df::DataFrame, net::PetriNet; tol = 1e-6)
    u_last = [df[end, String(net.place_names[i])] for i = 1:net.n_places]
    for j = 1:net.n_transitions
        can_fire = true
        for i = 1:net.n_places
            if net.incidence[i, j] < 0 && u_last[i] < -net.incidence[i, j] - tol
                can_fire = false
                break
            end
        end
        if can_fire
            return false
        end
    end
    return true
end

# Визуализация

function plot_marking_evolution(df::DataFrame, N::Int)
    plots = []
    for group in ["Think", "Hungry", "Eat", "Fork"]
        p = plot(xlabel = "Time", ylabel = group, title = "$group states")
        for i = 1:N
            col = "$group"_$i
            if col in names(df)
                plot!(df.time, df[!, col], label = "$group"_$i)
            end
        end
        push!(plots, p)
    end
    return plot(plots..., layout = (4, 1), size = (800, 1000))
end

end # module


^G Help       ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo
```

Рисунок 3.2: код

3.2 Базовый эксперимент

Создаю папки plots data scripts и файл dining_philosophers.jl (рис. 3.3).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ mkdir -p plots data

Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ mkdir -p scripts

Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ nano scripts/dining_philosophers.jl
```

Рисунок 3.3: создание папок и файла

В открывшемся окне вставила код из методички (рис. 3.4).

```
GNU nano 8.7                    scripts/dining_philosophers.jl
# scripts/dining_philosophers.jl

using DrWatson
using DataFrames, CSV, Plots

include(joinpath(@__DIR__, "..", "src", "DiningPhilosophers.jl"))
using .DiningPhilosophers

N = 5
tmax = 50.0

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(joinpath("data", "dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(joinpath("plots", "classic_simulation.png"))

println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(joinpath("data", "dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(joinpath("plots", "arbiter_simulation.png"))

println("\n Готово! Результаты сохранены в data/ и plots/")
```

Рисунок 3.4: код

Установила необходимые пакеты (рис. 3.5).

```
      Info Packages marked with ^ and ⌅ have new versions available. Those with ^ may be upgradable,
ut those with ⌅ are restricted by compatibility constraints from upgrading. To see why use `status --o
tdated -m`
recompiling packages ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 82/127
  NonlinearSolveSpectralMethods
  NonlinearSolveQuasiNewton
  NonlinearSolveFirstOrder
  SimpleNonlinearSolve
  DiffEqBase → DiffEqBaseChainRulesCoreExt
```

Рисунок 3.5: установка пакетов

Запустила код и получила результаты (рис. 3.6).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/stud
b5
$ julia scripts/dining_philosophers.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Deadlock обнаружен: false

 Готово! Результаты сохранены в data/ и plots/
```

Рисунок 3.6: запуск

График классической симуляции (рис. 3.7).

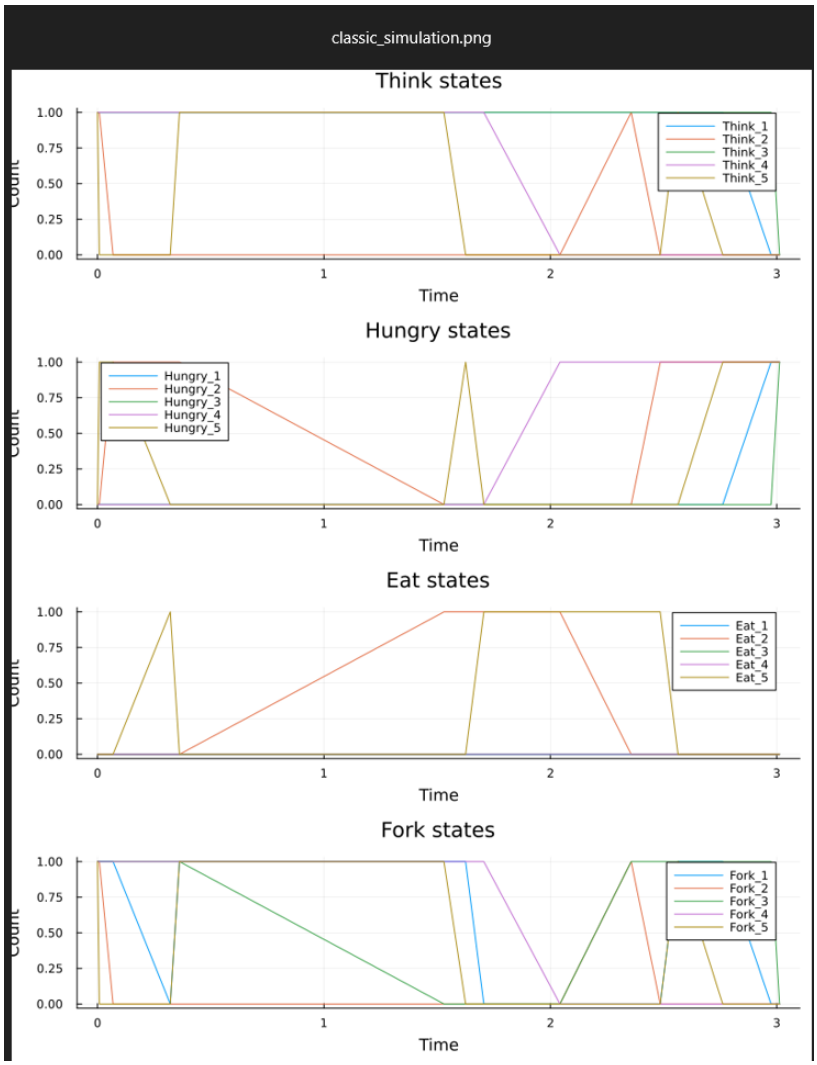

Рисунок 3.7: график1

График арбитр симуляции (рис. 3.8).

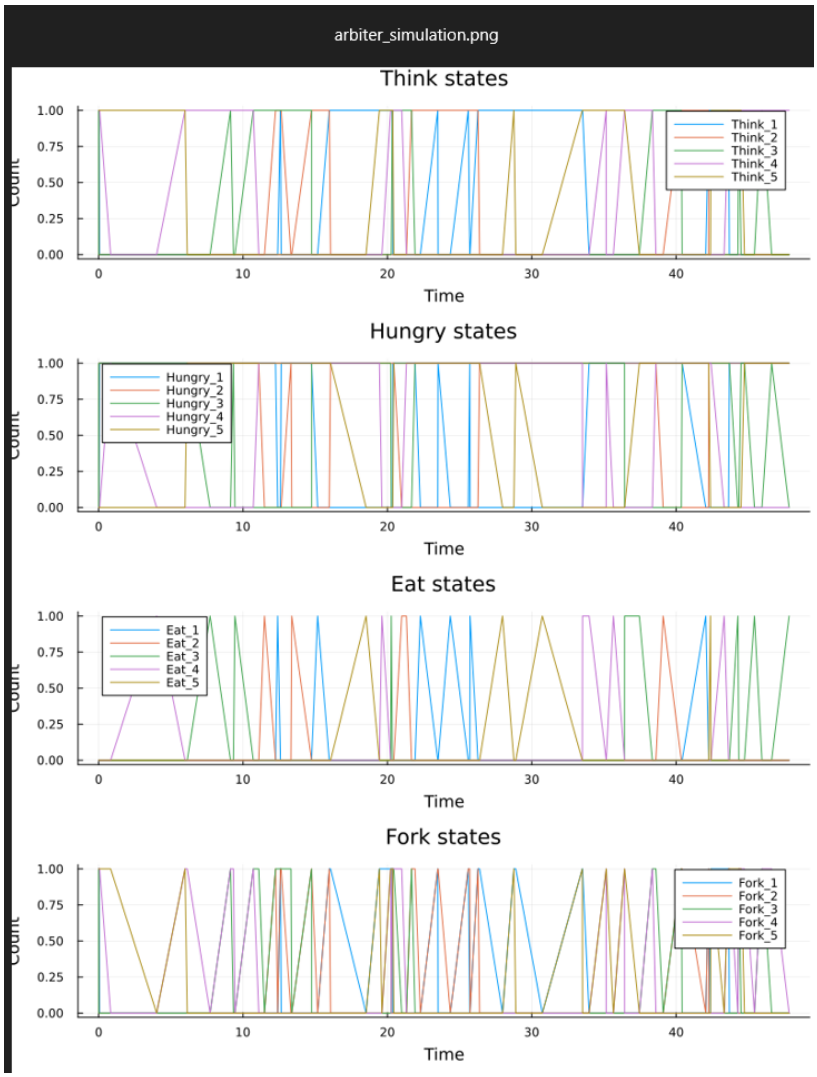

Рисунок 3.8: график2

3.3 Анимация процесса

Создаю файл dining_philosophers_animation.jl (рис. 3.9).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ nano scripts/dining_philosophers_animation.jl
```

Рисунок 3.9: файл

В открывшемся окне вставила код из медодички (рис. 3.10).

```
  GNU nano 8.7                    scripts/dining_philosophers_animation.jl
# scripts/dining_philosophers_animation.jl

using DrWatson
using DataFrames, CSV, Plots, Random

include(joinpath(@__DIR__, "..", "src", "DiningPhilosophers.jl"))
using .DiningPhilosophers

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )
    xticks!(1:length(u), string.(names), rotation = 45)
end

gif(anim, joinpath("plots", "philosophers_simulation.gif"), fps = 2)
println("Анимация сохранена в plots/philosophers_simulation.gif")
```

Рисунок 3.10: код

Установила необходимые пакеты (рис. 3.11).

```
(@v1.12) pkg> add OrdinaryDiffEq Plots DataFrames Random LinearAlgebra CSV DrWatson
[ Info: Use `./Plots` to add or develop the local directory at `C:\Users\Polina Samarkhanova\work\study
\2026-1\2026-1==study--simulation-modeling\labs\lab5\Plots`.
   Resolving package versions...
     Project No packages added to or removed from `C:\Users\Polina Samarkhanova\.julia\environments\v1.
12\Project.toml`
    Manifest No packages added to or removed from `C:\Users\Polina Samarkhanova\.julia\environments\v1.
12\Manifest.toml`
```

Рисунок 3.11: установка пакетов

Запустила код и получила результат - анимацию (рис. 3.12).

```
Polina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ julia scripts/dining_philosophers_animation.jl
[ Info: Saved animation to C:\Users\Polina Samarkhanova\work\study\2026-1\2026-1==study--simulation-mod
eling\labs\lab5\plots\philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
```

Рисунок 3.12: запуск

Просмотр результата(скрин анимации - видео на платформах) (рис. 3.13).

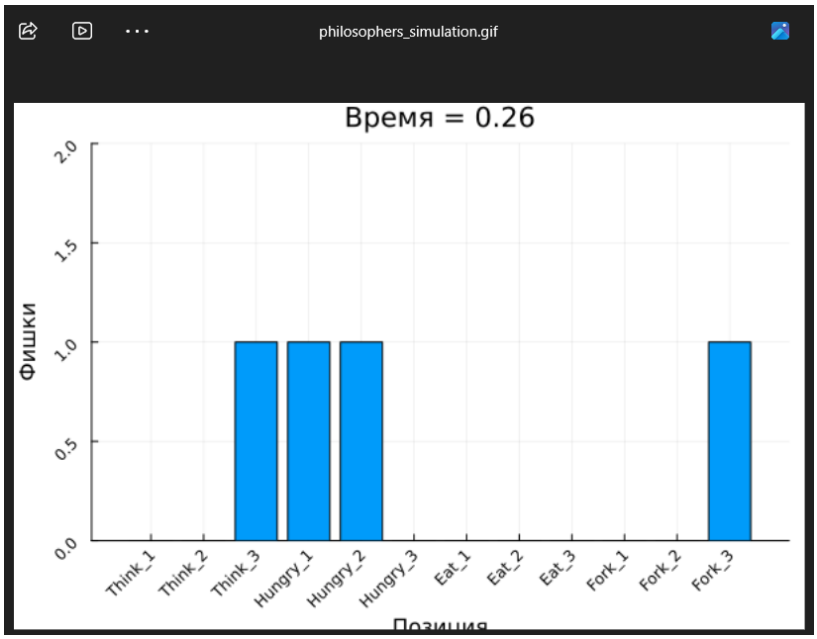

Рисунок 3.13: анимация

3.4 Итоговый отчет

Создала файл dining_philosophers_report.jl (рис. 3.14).

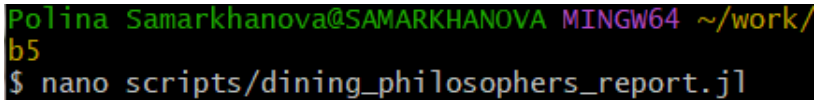

Рисунок 3.14: файл

В открывшемся окне вставила код из медодички (рис. 3.15).

```
  GNU nano 8.7                    scripts/dining_philosophers_report.jl
# scripts/dining_philosophers_report.jl

using DrWatson
using DataFrames, CSV, Plots

df_classic = CSV.read(joinpath("data", "dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(joinpath("data", "dining_arbiter.csv"), DataFrame)
N = 5

# Столбцы для состояния "Ест"
eat_cols = [Symbol("Eat_$i") for i = 1:N]

p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["Ф $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)

p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["Ф $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
)

p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(joinpath("plots", "final_report.png"))

println("Отчёт сохранён в plots/final_report.png")
```

Рисунок 3.15: код

Запустила код и получила результат (рис. 3.16).

```
olina Samarkhanova@SAMARKHANOVA MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/labs/la
b5
$ julia scripts/dining_philosophers_report.jl
Отчёт сохранён в plots/final_report.png
```

Рисунок 3.16: запуск

Открыла графики с результатом (рис. 3.17).

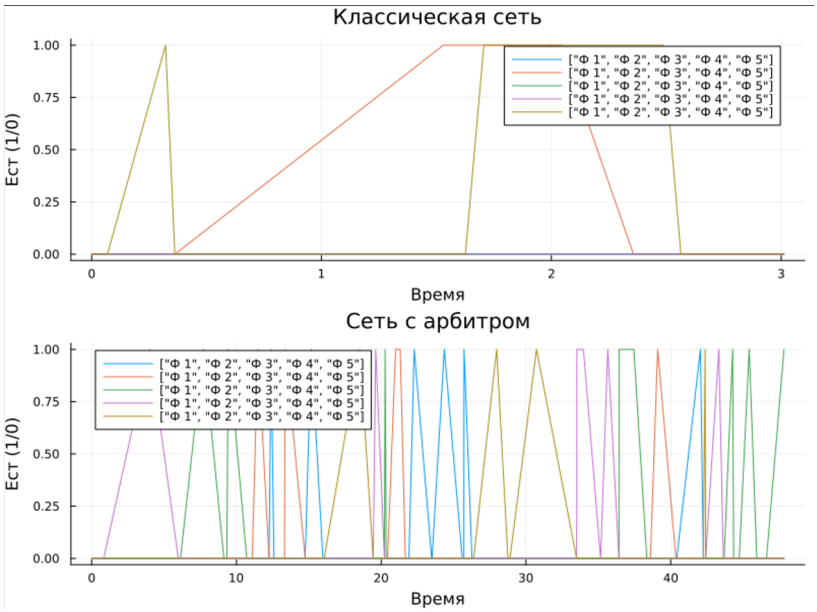

Рисунок 3.17: графики

4 Выводы

В ходе лабораторной работы реализованы две модели сети Петри для задачи обедающих философов. Классическая модель приводит к deadlock (все философы голодают), что подтверждается стохастическим моделированием и визуализацией. Модифицированная модель с арбитром успешно предотвращает взаимную блокировку, демонстрируя эффективный способ синхронизации. Анимация и итоговые графики наглядно показывают разницу в поведении двух моделей.